

```

# -----
# Back-End Tab Integration (Handling Text Highlights)
# To coordinate this dialog with an individual tab containing a QPlainTextEdit
# you must catch the find_requested signal. PySide6 manages document
# highlighting using QTextDocument.find().
# -----

from PySide6.QtWidgets import QPlainTextEdit
from PySide6.QtGui import QTextDocument, QTextCursor
from PySide6.QtCore import QEvent, Qt

from TabFindDialog import TabFindDialog

class EditorTab(QPlainTextEdit):
    def __init__(self, parent=None):
        super().__init__(parent)

        self.setReadOnly(True)
        # Enable text selection via mouse and keyboard
        self.setTextInteractionFlags(Qt.TextInteractionFlag.TextSelectableByMouse |
                                    Qt.TextInteractionFlag.TextSelectableByKeyboard
                                    )

        self.current_match_idx = 0
        self.total_matches = 0
        self.last_search_term = ""
        self.last_case_setting = False
        self.find_dialog = None

    def toggle_find_dialog(self):
        if not self.find_dialog:
            self.find_dialog = TabFindDialog(self.viewport())
            self.find_dialog.find_requested.connect(self.handle_find)
            self.find_dialog.closed.connect(self.on_dialog_closed)
            self.viewport().installEventFilter(self)

        if self.find_dialog.isVisible():
            self.find_dialog.hide()
        else:
            self.find_dialog.show()
            self.reposition_find_dialog()
            self.find_dialog.search_input.setFocus()
            self.find_dialog.search_input.selectAll()

    def reposition_find_dialog(self):
        if self.find_dialog and self.find_dialog.isVisible():
            view_rect = self.viewport().rect()
            dialog_width = self.find_dialog.width()
            x = view_rect.right() - dialog_width - 20
            y = view_rect.top() + 10
            self.find_dialog.move(x, y)
            self.find_dialog.raise_()

    def eventFilter(self, watched, event):
        if watched == self.viewport():
            if event.type() in (QEvent.Type.Resize, QEvent.Type.Paint, QEvent.Type.UpdateRequest):
                self.reposition_find_dialog()
        return super().eventFilter(watched, event)

    def handle_find(self, text: str, forward: bool, case_sensitive: bool):
        # FIXED: Forces an instantaneous match count update when configuration states pivot
        if text != self.last_search_term or case_sensitive != self.last_case_setting:
            self.calculate_match_counts(text, case_sensitive)
            self.last_search_term = text
            self.last_case_setting = case_sensitive

        # FIXED: Guard block instantly outputs "0 of 0" if lookahead evaluation catches zero items
        if self.total_matches == 0:
            self.current_match_idx = 0
            if self.find_dialog:
                self.find_dialog.update_counter(0, 0)
            return

        flags = QTextDocument.FindFlag(0)
        if not forward:
            flags |= QTextDocument.FindFlag.FindBackward
        if case_sensitive:
            flags |= QTextDocument.FindFlag.FindCaseSensitively

        found = self.find(text, flags)

        if not found:
            cursor = self.textCursor()
            cursor.movePosition(QTextCursor.MoveOperation.Start if forward else QTextCursor.MoveOperation.End)
            self.setTextCursor(cursor)
            found = self.find(text, flags)

```

```

        if found:
            self.update_current_match_index(forward)

        # FIXED: Counter label updates after computing index mutations
        if self.find_dialog:
            self.find_dialog.update_counter(self.current_match_idx, self.total_matches)

def calculate_match_counts(self, text: str, case_sensitive: bool):
    doc = self.document()
    flags = QTextDocument.FindFlag.FindCaseSensitively if case_sensitive else QTextDocument.FindFlag(0)

    count = 0
    cursor = doc.find(text, 0, flags)
    while not cursor.isNull():
        count += 1
        cursor = doc.find(text, cursor, flags)

    self.total_matches = count
    self.current_match_idx = 0

def update_current_match_index(self, forward: bool):
    if self.total_matches == 0:
        self.current_match_idx = 0
        return
    if forward:
        self.current_match_idx = (self.current_match_idx % self.total_matches) + 1
    else:
        self.current_match_idx = self.total_matches if self.current_match_idx <= 1 else self.current_match_idx - 1

def on_dialog_closed(self):
    if self.viewport():
        self.viewport().removeEventFilter(self)
    self.find_dialog = None

def jump_to_coordinates(self, line: int, column: int, absolute_position: int = None, is_one_indexed: bool = True):
    """
    Moves the text cursor precisely to the given token coordinates.
    Uses absolute character targeting to maintain flawless alignment
    regardless of text wrapping configurations or carriage conversions.
    """
    self.setFocus()
    doc = self.document()
    if doc.blockCount() == 0:
        return

    # 1. OPTIMIZED ANCHOR JUMP: If the raw absolute character index is available, use it!
    # This completely neutralizes word-wrap line distortions and character stripping errors.
    if absolute_position is not None:
        # Bound-clip the index position safely within the document length
        safe_pos = max(0, min(absolute_position, doc.characterCount() - 1))

        cursor = QTextCursor(doc)
        cursor.setPosition(safe_pos)

        # Extract the actual block context where this character landed
        block = cursor.block()
    else:
        # Fallback block matching if absolute_position wasn't passed down
        target_line = (line - 1) if is_one_indexed else line
        target_line = max(0, min(target_line, doc.blockCount() - 1))
        block = doc.findBlockByLineNumber(target_line)
        if not block.isValid():
            return
        cursor = QTextCursor(block)
        line_length = len(block.text())
        safe_col = max(0, min((column - 1) if is_one_indexed else column, line_length))
        cursor.movePosition(QTextCursor.MoveOperation.Right, QTextCursor.MoveMode.MoveAnchor, safe_col)

    # 2. Apply Visual Highlighting Box over the macro keyword phrase
    if block.isValid():
        line_text = block.text()
        pos_in_block = cursor.positionInBlock()
        remaining_text = line_text[pos_in_block:]

        if "\\index" in remaining_text:
            idx_offset = remaining_text.find("\\index")
            cursor.movePosition(QTextCursor.MoveOperation.Right, QTextCursor.MoveMode.MoveAnchor, idx_offset)
            closing_brace_idx = remaining_text.find("}", idx_offset)
            highlight_length = (closing_brace_idx - idx_offset + 1) if closing_brace_idx != -1 else 7
            cursor.movePosition(QTextCursor.MoveOperation.Right, QTextCursor.MoveMode.KeepAnchor, highlight_length)
        else:
            cursor.movePosition(QTextCursor.MoveOperation.Right, QTextCursor.MoveMode.KeepAnchor, min(10, len(remaining_text)))

    # 3. Commit the structural changes back to the text viewport layout
    self.setTextCursor(cursor)
    self.ensureCursorVisible()

```

```
self.centerCursor()

def keyPressEvent(self, event):
    # Check if the pressed key is Escape
    if event.key() == Qt.Key_Escape:
        cursor = self.textCursor()
        # Clear selection by setting the anchor to the current cursor position
        cursor.clearSelection()
        self.setTextCursor(cursor)
        # Accept the event so it doesn't propagate further
        event.accept()
    else:
        # Pass all other key events to the default QPlainTextEdit handler
        super().keyPressEvent(event)
```